

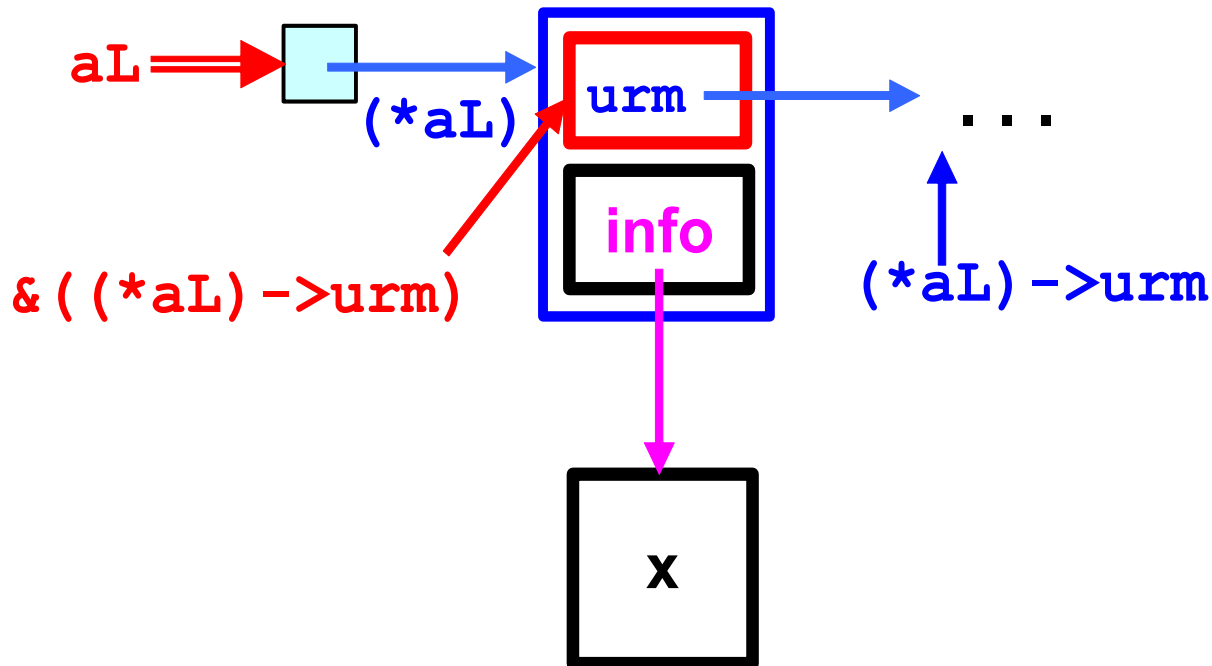
Tipul lista generica

- Pot exista insa situatii in care in aceeaasi aplicatie se lucreaza cu mai multe **liste, cu elemente de tipuri diferite** (de exemplu lista de materii si liste de studenti).
- **operatiile de baza asupra listelor sunt aceleasi**, dar **modul de tratare a partii de informatie difera**, fiind strict legat de tipul elementelor din lista prelucrata la un moment dat.
- Pentru a rezolva aceste situatii vom defini tipul “**lista generica**”, care face abstractie de tipul elementelor concrete.

Cum se memoreaza elementele unei liste generice?

Intr-o zona de memorie gestionata de utilizator

→ la crearea celulei trebuie cunoscuta **adresa** zonei in care se gaseste valoarea elementului



Definirea tipului lista generica

Adresele necesare sunt adresele unor elemente de tip necunoscut in momentul definirii tipului lista generica, deci vor fi declarate **void***, urmand ca la nivelul aplicatiei sa fie convertite la tipul corespunzator.

Campul info este adresa unui element de tip specific aplicatiei, deci de tip **void***.

Definirea tipului lista generica

```
typedef struct celulag
{ void *info;          /* informatie */
  struct celulag *urm; /* adresa urmatoarei celule */
} TCelulaG, *TLG;

/* tipuri: Celula, Lista */
```

Adresarea la nivelul aplicatiei

Sa consideram:

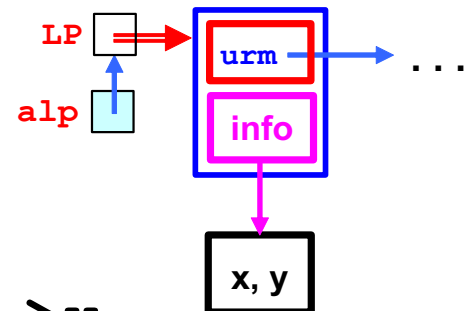
```
typedef struct { int x, y; } TPunct;
                /* punct in plan */
TLG LP = NULL; /* lista puncte */
ALG alp = &LP;
```

Expresia abscisei primului punct din lista este:

`((TPunct*) (LP->info)) ->x`

sau

`((TPunct*) ((*alp) ->info)) ->x`



Inserarea in liste generice

Operatia de inserare are trei etape:

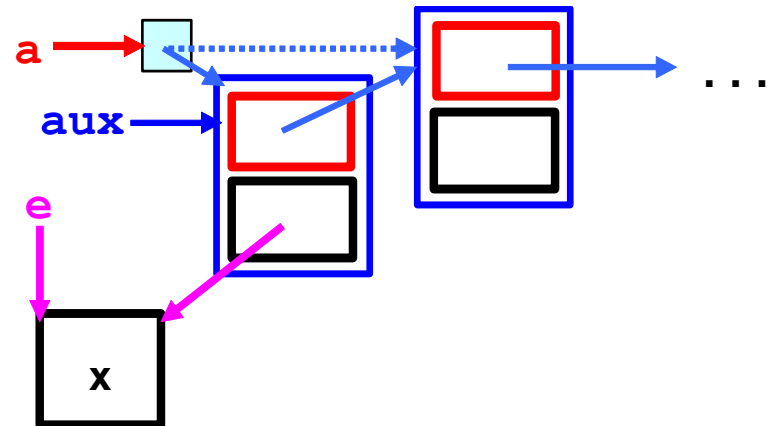
1. **alocarea** de spatiu pentru o noua celula
2. **completarea** campului **info**
3. **conectarea** celulei in lista.

Dimensiunea noii celule este:

`sizeof(TCelulaG)`

Inserarea in liste generice

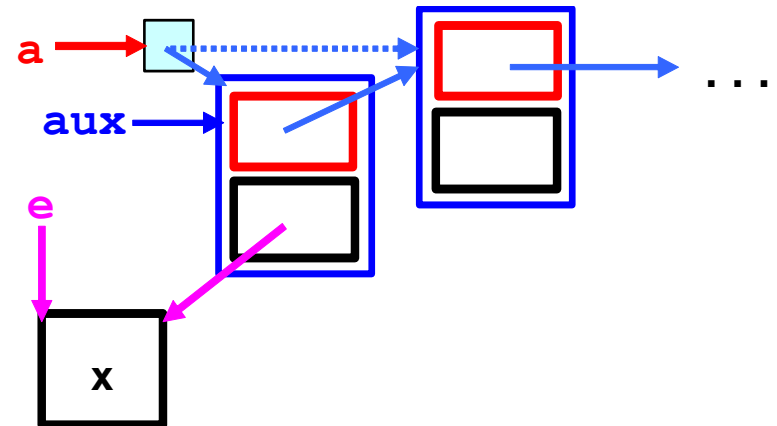
```
int InsLG(ALG aL, void* ae, size_t d)
{
    TLG aux = (TLG)malloc(sizeof(TCelulaG));
    if(!aux) return 0;
    aux->info = malloc(d);
    if (!aux->info)
    { free(aux); return 0;}
    memcpy(aux->info, ae, d);
    aux->urm = *aL;
    *aL = aux;
    return 1;
}
```



Inserarea in liste generice

O alta varianta:

```
int InsLG(ALG aL, void* ae)
{
    TLG aux = (TLG)malloc(sizeof(TCelulaG)) ;
    if(!aux) return 0;
    aux->info = ae;
    aux->urm = *aL;
    *aL = aux;
    return 1;
}
```



Distrugge lista generica

```
void DistrugeLG(ALG aL) /* eliminare celula si element */  
{ TLG aux;  
    while(*aL)  
    {  
        aux = *aL;  
        *aL = aux->urm;  
  
        free(aux->info. /* sau utilizare functie  
                        de eliminare */  
        free(aux);  
    }  
}
```

Prelucrare lista de pointeri la puncte din plan

```
void Afisare(ALG aL)
{ TPunct p;
  if(!*aL) { printf("Lista vida\n"); return;}
  printf("Lista: ");
  for(; *aL; aL = &(aL)->urm)
  {
    p = *(TPunct*) ((*aL)->info);
    printf("[%d %d] ", p.x, p.y);
  }
  printf("\n");
}
```